

Jenkins

[Jenkins](#), es un servidor de automatización de código abierto utilizado para implementar procesos de integración continua y entrega continua (CI/CD) en el desarrollo de software.

A continuación se presentan 5 [preguntas clave sobre Jenkins](#) basadas en entrevistas y conceptos esenciales:

1. ¿Qué es Jenkins?

Es una herramienta escrita en Java que actúa como servidor de automatización independiente. Permite a los desarrolladores construir, probar y desplegar sus proyectos de software de manera confiable y automatizada.

- [Mas Info](#)
- [Mas Info](#)
- [Mas Info](#)
- [Mas Info](#)

2. ¿Para qué se utiliza principalmente? Se utiliza para la Integración Continua (**CI**) y la Entrega Continua (**CD**). Detecta cambios en el código fuente (como un commit en un repositorio de Git), compila la aplicación, ejecuta pruebas automatizadas y reporta los resultados sin intervención manual constante.

- [Mas Info](#)
- [Mas Info](#)
- [Mas Info](#)

3. ¿Qué son los Plugins en Jenkins? Son complementos de terceros que extienden las funciones principales del servidor. Permiten integrar herramientas de control de versiones (Git, GitHub), sistemas de notificación (Slack), gestores de compilación o herramientas de pruebas (JUnit, JMeter).

- [Mas Info](#)
- [Mas Info](#)
- [Mas Info](#)

4. ¿Qué es un Pipeline en Jenkins? Es una combinación de plugins que apoyan la integración y la implementación de un flujo de entrega continua mediante código. Se define comúnmente como un conjunto de instrucciones (Pipeline as Code) que describe todas las etapas por las que pasa el software, desde el código fuente hasta la producción.

- [Mas Info](#)
- [Mas Info](#)
- [Mas Info](#)

5. ¿Qué son los agentes (agents) en Jenkins? Son máquinas o nodos independientes configurados para ejecutar los trabajos (jobs) asignados por el servidor principal (Master). Permiten distribuir la carga de trabajo (distributed builds) en distintos entornos o sistemas operativos para acelerar el proceso de compilación y prueba.

- [Mas Info](#)
- [Mas Info](#)

Una experiencia real

La experiencia real con [Jenkins](#) en entornos de producción combina una gran potencia de automatización con importantes retos de mantenimiento de infraestructura.

[DonWeb](#)

[Video](#)

[Primer Pipeline](#)

Lo positivo en el día a día

- **Flexibilidad total:** Al ser open-source y autohospedado, puedes adaptarlo a cualquier infraestructura, por compleja o aislada que esté.
- **Ecosistema de Plugins:** Permite conectar prácticamente cualquier herramienta heredada o moderna mediante su inmenso catálogo.
- **Control de flujos con Jenkinsfile:** Definir la integración continua (CI) como código al lado del repositorio facilita la trazabilidad de las fases de build, tests y empaquetado. Como se señala en este análisis de casos reales, el compromiso del equipo con el control de versiones es vital para que esta automatización funcione.
 - [Mas Info](#)
 - [Video](#)
 - [Jenkins vs Github Actions](#)
 - [Primer Pipeline con Roxs](#)

Los dolores de cabeza habituales

- **Mantenimiento del servidor:** Administrar actualizaciones de plugins, dependencias de Java y la seguridad del nodo principal (controller) consume tiempo de operaciones.
 - [Video](#)
 - [Jenkins vs Github Actions](#)
- **Gestión de agentes:** Configurar y escalar nodos o agentes (por ejemplo, vía puertos TCP o contenedores) puede volverse complejo y propenso a bloqueos de red si no se dimensiona bien.
 - [Video](#)
 - [Video](#)
- **Deuda técnica en los pipelines:** Con el tiempo, la acumulación de ramas temporales y scripts mal estructurados dificulta el mantenimiento, un problema muy común debatido por usuarios en foros de experiencias reales en Reddit.
 - [Real World Examples](#)
 - [Se puede ver una introducción práctica al funcionamiento de los flujos de trabajo aquí](#)

Github Actions

GitHub Actions es una plataforma de automatización e integración y despliegue continuos (CI/CD) que te permite ejecutar tareas automáticas directamente en tu repositorio de GitHub.

En palabras sencillas: sirve para crear "recetas" o flujos de trabajo que se activan solos cuando pasa algo en tu código (por ejemplo, cuando subes cambios, abres un Pull Request o creas una nueva versión).

¿Para qué se utiliza?

- **Pruebas automáticas (CI):** Cada vez que tú o tu equipo suban código, GitHub Actions puede ejecutar los tests para asegurarse de que nada se haya roto.
 - [Que es Github Actions](#)
- **Despliegue automático (CD):** Si las pruebas pasan con éxito, la plataforma puede enviar el nuevo código directamente a servidores de producción como AWS, Azure, Google Cloud o Vercel.
 - [Video](#)
- **Gestión del repositorio:** Automatizar tareas administrativas como poner etiquetas a los reportes de errores (Issues), cerrar comentarios antiguos o dar la bienvenida a nuevos colaboradores.
 - [Video](#)
 - [Features](#)

Componentes clave de GitHub Actions

Para entender cómo funciona, imagina una [línea de producción](#) dividida en varias partes:

- **Workflows (Flujos de trabajo):** Son los procesos automatizados globales. Se configuran mediante un archivo de texto en formato YAML dentro de la carpeta `.github/workflows/` de tu proyecto.
 - [Video](#)
 - [Video](#)
- **Events (Eventos):** Es el detonante que arranca el flujo. Ejemplos: un push de código, un pull_request, o incluso una hora programada (como un cron job).
 - [Video](#)
 - [Video](#)
- **Jobs (Trabajos):** Un conjunto de pasos que se ejecutan en una misma máquina virtual. Un flujo puede tener varios jobs corriendo al mismo tiempo o uno detrás de otro.
 - [Video](#)
 - [Video](#)
- **Steps (Pasos):** Tareas individuales dentro de un job. Puede ser algo tan simple como ejecutar un comando de terminal (ej. `npm test`) o usar una "Action" preestablecida.
 - [Sintaxis Básica](#)
 - [Video](#)
 - [Video](#)
- **Actions (Acciones):** Son bloques de código ya programados y listos para usar (ya sea creados por

GitHub o por la comunidad en su Marketplace de GitHub). Te ahorran tener que escribir toda la lógica desde cero para tareas comunes, como configurar Node.js o conectarte a un servidor.

- [Features](#)
- [Sintaxis Básica](#)
- [Video](#)
- **Runners (Ejecutores):** Son los servidores de GitHub (que pueden usar Linux, Windows o macOS) donde se descargará tu código y se ejecutarán los flujos de trabajo.
 - [Understanding](#)
 - [Video](#)

Ejemplo básico de un archivo de configuración

(.github/workflows/test.yml)

```
name: Mi Primer Flujo de Trabajo

# 1. El evento: Se activa cada vez que alguien hace "push" a la rama main
on:
  push:
    branches: [ main ]

# 2. Los trabajos a realizar
jobs:
  ejecutar-pruebas:
    # Corre sobre un servidor Linux virtual provisto por GitHub
    runs-on: ubuntu-latest

    # 3. Los pasos consecutivos
    steps:
      # Descarga el código del repositorio en el servidor virtual
      - name: Descargar código
        uses: actions/checkout@v4

      # Configura el entorno de Node.js
      - name: Configurar Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      # Instala las dependencias y corre los tests de tu app
      - name: Instalar e iniciar pruebas
        run: |
          npm install
          npm test
```

Ventajas principales

- **Integración total:** No necesitas configurar herramientas externas (como Jenkins o Travis CI) porque ya está integrado en el propio ecosistema de GitHub.
- **Gratuito para públicos:** Es completamente gratis para repositorios públicos, y ofrece una generosa cantidad de minutos mensuales gratuitos para repositorios privados.
 - [Video](#)
 - [Video](#)
 - [Understanding](#)